

Internal Reference Document

PulseCX

Customer Experience Analytics Platform

A Guide to the Architecture, Code and Technologies Behind the Project

Prepared for onboarding and self-study | Covers `app.py`, `models.py`, templates and static assets

Table of Contents

1. Introduction & Purpose
2. System Architecture
3. Technology Stack Explained
4. Project Structure
5. Database Design (models.py)
6. Application Bootstrap (app.py setup)
7. The Sentiment & Emotion Engine
8. The AI Insight Generator
9. Authentication & Security
10. Feature Walkthrough by Route
11. Frontend Layer (Templates, Tailwind, Chart.js)
12. PDF Report Pipeline
13. Running & Configuring the Project
14. API Quick Reference
15. Glossary of Terms
16. Suggested Learning Path

1. Introduction & Purpose

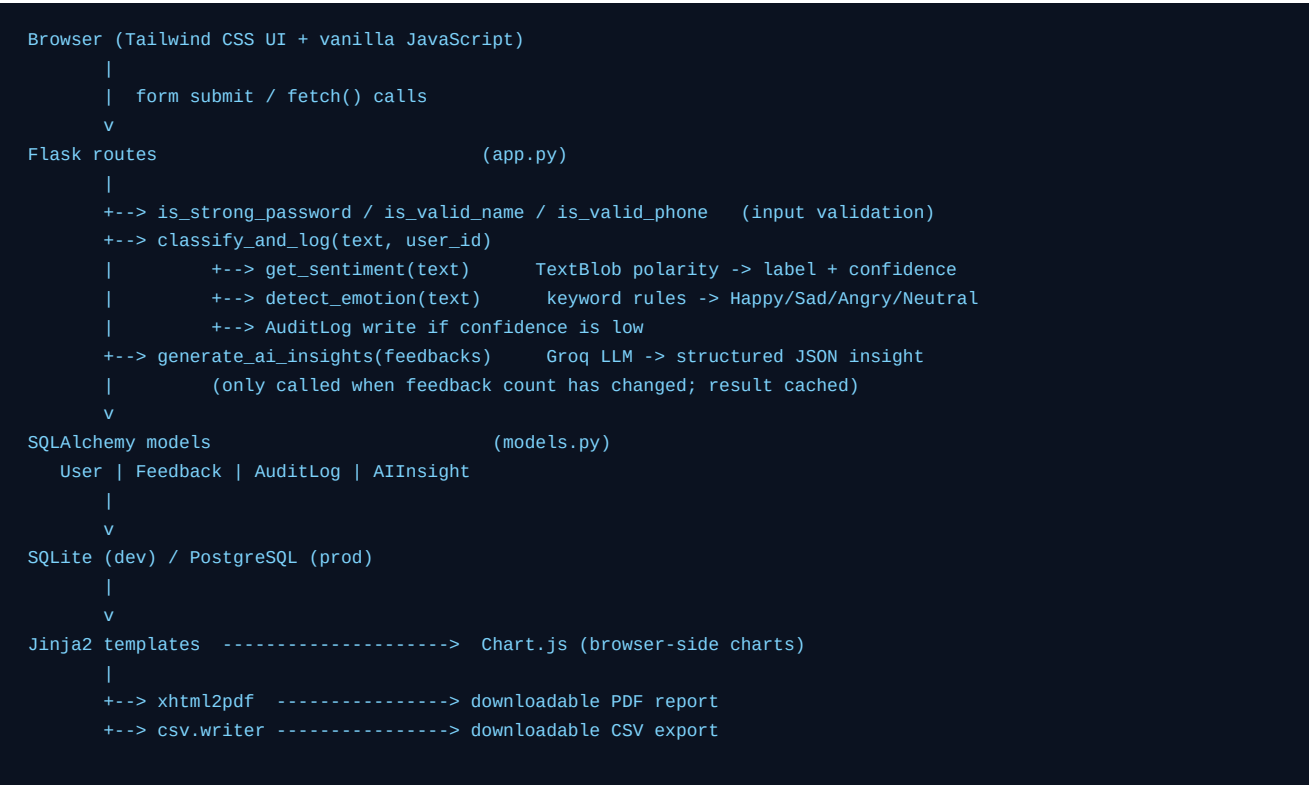
PulseCX is a full-stack web application that collects customer feedback and turns it into sentiment analytics, emotion breakdowns, and business-facing reports. A user can type feedback into a form, upload a CSV of many rows at once, or submit it through a small JSON API. Every piece of feedback is automatically classified for **sentiment** (Positive / Negative / Neutral) and **emotion** (Happy / Sad / Angry / Neutral), stored in a database, and rolled up into dashboards, trend charts, and downloadable reports.

The project is built with **Flask**, a lightweight Python web framework, and uses a combination of a classic statistical NLP technique (TextBlob polarity scoring) for real-time classification and a hosted large language model (Grok's LLaMA 3.3 70B) for higher-level, narrative business insights. This split matters: the fast, cheap classifier runs on every keystroke and every saved row, while the expensive LLM call only runs when the underlying data actually changes, with its output cached in the database.

This document walks through the codebase top to bottom — the data model, the request handling in `app.py`, the classification and insight-generation logic, the security model, and the frontend — so that someone new to the project can understand not just *what* the code does, but *why* it is structured the way it is.

2. System Architecture

At a high level, a request flows through four layers: the browser, the Flask route, the processing/analysis layer, and the database. Read/report pages then recombine data from the database back out through Jinja2 templates or a generated PDF.



A key design decision is that **every persisted feedback row goes through exactly one function, `classify_and_log()`**. Manual entry, CSV import, the JSON API, and edits all call it. That guarantees the same classifier version and the same low-confidence logging behavior everywhere, rather than having each entry point reimplement classification slightly differently. The one exception is the live-typing preview endpoint (`/live_sentiment`), which intentionally calls the lower-level `get_sentiment()` directly — logging every keystroke of an unfinished sentence to the audit log would be noise, not signal.

3. Technology Stack Explained

Backend

Technology	Role in this project
------------	----------------------

Flask	The web framework. Defines routes (URLs), handles requests/responses, and renders HTML templates.
Flask-SQLAlchemy	Object-Relational Mapper (ORM) — lets the code work with Python classes (<code>User</code> , <code>Feedback</code> , ...) instead of writing raw SQL.
Flask-Migrate (Alembic)	Generates and applies versioned database schema changes ("migrations") so the schema can evolve without hand-written SQL scripts.
Flask-Login	Session-based authentication: tracks who is logged in, protects routes with <code>@login_required</code> .
Flask-WTF (CSRFProtect)	Cross-Site Request Forgery protection on all state-changing form submissions.
Flask-Mail	Sends the password-reset email over SMTP.
Flask-Compress	Gzip-compresses HTML/JSON/CSS/JS responses to cut transfer size.
Werkzeug	Provides <code>generate_password_hash/check_password_hash</code> (PBKDF2) and <code>secure_filename</code> for safe file uploads.

Analysis & AI

Technology	Role in this project
TextBlob	A lightweight NLP library used for sentiment polarity scoring — the primary, always-on classifier.
scikit-learn	Supplies <code>accuracy_score/precision_score/recall_score</code> , used to build the self-consistency widget on the dashboard (see Section 7).
Groq API (LLaMA 3.3 70B)	A hosted large language model called once per data change to turn aggregate statistics and sample feedback text into a structured business summary.

Data & Reporting

Technology	Role in this project
SQLite / PostgreSQL	The relational database. SQLite by default (a single file, zero setup); PostgreSQL in production via <code>DATABASE_URL</code> .
matplotlib	Renders the sentiment donut chart as a PNG image (base64-encoded) for embedding directly into the PDF report.
xhtml2pdf	Converts a rendered HTML template into a PDF file — used for the downloadable report.

Chart.js	Renders interactive charts (trend lines, donut charts) in the browser on the dashboard and analytics pages.
----------	---

Frontend

Technology	Role in this project
Jinja2	Flask's template engine — renders Python data into HTML (<code>templates/*.html</code>).
Tailwind CSS	Utility-class based styling for layout, color, spacing and the glassmorphism-style UI.
Vanilla JavaScript	No frontend framework; <code>static/js/*.js</code> handles the live-typing preview, KPI counters, theme toggling, and custom select components directly with the DOM API.
lucide / three.js	Self-hosted vendor libraries under <code>static/vendor/</code> for icons and background visual effects.

4. Project Structure

```
customer-experience-CX--main/
|-- app.py           Flask app: routes, auth, classification, insight engine, reports
|-- models.py        SQLAlchemy models: User, Feedback, AuditLog, AIInsight
|-- requirements.txt  Python dependencies
|-- .env.example      Template for required environment variables
|-- migrations/       Alembic migration scripts (versions/*.py)
|-- templates/        Jinja2 HTML templates
|  |-- base.html      Shared layout: nav, sidebar, theme toggle
|  |-- _macros.html    Reusable template snippets (e.g. KPI card)
|  |-- landing.html    Public marketing/landing page
|  |-- login.html / register.html / forgot_password.html / reset_password.html
|  |-- dashboard.html  Main authenticated view: entry form, KPIs, AI insight
|  |-- analytics.html  Charts and breakdowns
|  |-- customers.html  Searchable/filterable feedback table
|  |-- reports.html / report_pdf.html  On-screen report + printable PDF version
|  |-- admin.html      Admin-only user/feedback management
|  |-- settings.html   Account settings
|  |-- edit.html / about.html / error.html
|-- static/
|  |-- js/            custom-select.js, kpi-count.js, theme.js
|  |-- vendor/         chart.umd.min.js, lucide.min.js, three.min.js (self-hosted, long-cached)
|  |-- style.css
`-- instance/         SQLite database file, created at runtime (not committed)
```

5. Database Design (models.py)

The schema is defined with four SQLAlchemy model classes. A shared `NAMING_CONVENTION` dict is passed into the `MetaData` object so that every index, unique constraint, and foreign key gets a predictable, named identity (e.g. `fk_feedbacks_user_id_users`) instead of an auto-generated anonymous one. This matters specifically for SQLite: Alembic's "batch mode" (rebuild-the-table-under-the-hood) migrations need to reference constraints by name to alter or drop them, and SQLite reflects unnamed constraints anonymously.

User

Field	Type	Notes
id	Integer	Primary key
username	String(100)	Unique, indexed
password	String(255)	PBKDF2 hash, never plaintext
role	String(20)	"user" or "admin", default "analyst" at the column level (routes set "user" explicitly on signup)
email	String(255)	Optional, unique, used for password reset
email_notify / ai_enabled	Boolean	Per-user feature toggles set from Settings
avatar	String(200)	Filename of the uploaded avatar image
reset_token / reset_token_expires	String / DateTime	Single active password-reset token per user, stored in the DB (not in memory) so it survives a restart

Feedback

Field	Type	Notes
name / phone / service	String	Optional submitter details; "Anonymous"/"General" defaults
text	Text	The feedback content itself
sentiment / sentiment_score	String / Float	Output of <code>get_sentiment()</code>
emotion / emotion_score	String / Float	Output of <code>detect_emotion()</code>
user_id	ForeignKey(users.id)	<code>onDelete="CASCADE"</code> — deleting a user removes their feedback

A composite index `ix_feedback_owner_created` on (`user_id`, `created_at`) speeds up the very common "this user's feedback, most recent first" query used throughout the dashboard, customers page, and reports.

AuditLog

Records notable system events — today, specifically low-confidence sentiment predictions (see Section 7). Each row stores which user, what action, what resource, a free-text detail string, and the requesting IP.

AllInsight

Caches the last LLM-generated insight per `(user_id, scope)` pair, where `scope` is either "my" (a user's own data) or "all" (the admin's site-wide view). A unique index on `(user_id, scope)` means there is at most one cached row per owner/scope combination; `feedback_count` is stored alongside so the app can tell whether the cache is stale (see `get_cached_insight()` in Section 8).

6. Application Bootstrap (app.py setup)

The top of `app.py`, before any route is defined, wires up the application:

- **Environment loading** — `load_dotenv()` reads `.env`. The app refuses to start if `GROQ_API_KEY` is missing (`raise ValueError(...)`), since the insight engine is a core feature, not an optional add-on.
- **CSRF protection** — `CSRFProtect(app)` is applied globally; JSON API routes that are called via `fetch()` without a form token (like `/live_sentiment`) opt out explicitly with `@csrf.exempt`.
- **Mail configuration** — read from environment variables. If `MAIL_USERNAME/MAIL_PASSWORD` are unset, the app still starts; it just logs a warning and the "forgot password" flow won't actually send email.
- **Static asset caching** — `SEND_FILE_MAX_AGE_DEFAULT` sets a 1-hour browser cache for regular static files; an `@app.after_request` hook additionally sets a 30-day `immutable` cache header specifically for `/static/vendor/`, since those third-party bundles are pinned by version and never change in place.
- **Database URL normalization** — some hosting providers hand out a `postgres://` URL, which SQLAlchemy's newer versions reject; the code rewrites it to `postgresql://`.
- **Connection pooling** — only configured for non-SQLite databases, since SQLite has no real concurrent connection pool to tune.
- **Foreign-key enforcement** — SQLite does not enforce `onDelete="CASCADE"` unless `PRAGMA foreign_keys=ON` is issued per connection; an SQLAlchemy `connect` event listener does this automatically for every new connection.
- **Gzip compression** — enabled via Flask-Compress if it is installed, wrapped in a `try/except ImportError` so its absence doesn't crash startup.
- **Centralized error handling** — 404/403/500 handlers check whether the request wants JSON (API paths, or an `Accept` header) and return either a JSON error body or a rendered `error.html` page.
- **Automatic migrations** — `sync_db_schema()` runs Alembic's `upgrade()` on every normal app startup (skipped when the process is itself an explicit `flask db ...` command), so a fresh checkout only needs `python app.py` to end up on the latest schema.
- **Login manager** — Flask-Login's `user_loader` callback tells it how to turn a stored session's user id back into a `User` object.

7. The Sentiment & Emotion Engine

`get_sentiment(text)`

Uses TextBlob's `.sentiment.polarity`, a float from -1 (very negative) to +1 (very positive). Two thresholds classify the label:

```
# app.py
POLARITY_POSITIVE = 0.1
POLARITY_NEGATIVE = -0.1

if polarity > POLARITY_POSITIVE:    label = "Positive"
elif polarity < POLARITY_NEGATIVE:  label = "Negative"
else:                               label = "Neutral"
```

Confidence is **not** a probability — TextBlob doesn't produce one. It is the distance the polarity sits from the nearest boundary, normalized to a 0–100 scale. A polarity of 0.35 is a confident Positive; a polarity of 0.11 is barely past the 0.1 boundary and is reported as low-confidence even though the label is still "Positive". This is what feeds the low-confidence audit logging described below.

`detect_emotion(text)`

A simple, transparent keyword-matching function: it lower-cases the text and checks for substrings like "happy"/"good"/"great" (→ Happy), "sad"/"bad"/"poor" (→ Sad), "angry"/"worst"/"frustrating" (→ Angry), else Neutral. It is intentionally simple and easy to extend with more keywords or replaced later with a trained classifier.

`classify_and_log(text, user_id)`

The single entry point that every *persisted* feedback row's classification goes through. It calls `get_sentiment()` and `detect_emotion()`, and if the sentiment confidence is below `LOW_CONFIDENCE_THRESHOLD` (30.0), it writes a `low_confidence_prediction` row to AuditLog so a human can review borderline calls later. Manual entry, CSV import, the JSON submit API, and edits all route through this one function — so "always uses the primary model, always logs low-confidence calls" is structural, not something each call site has to remember to do.

`evaluate_model(feedbacks)` — the self-consistency widget

This is explicitly **not** accuracy against labeled ground truth — there is no human-labeled validation dataset for this app's feedback. Instead it re-runs the current classifier against a sample of rows' already-stored text and measures how often the fresh result agrees with the sentiment that was saved at write time, using scikit-learn's `accuracy_score/precision_score/recall_score`. A low score means either the scoring logic has drifted since those rows were classified, or a lot of the text sits right on a decision boundary. The result is cached per (`owner`, `scope`) keyed on the row count, the same invalidation pattern used for AI insights, so the dashboard's 30-second auto-refresh does not re-run this analysis on every single poll.

8. The AI Insight Generator

This is the feature that turns raw feedback rows into an executive-style narrative: a summary, pain points, positive highlights, recommendations, action items, a trend read, and a short predictive note.

How the prompt is built

`summarize_feedback_stats()` computes aggregate numbers (totals, sentiment split, CSAT percentage, emotion counts, top services). `generate_ai_insights()` then takes up to 10 negative samples, 10 positive samples, and 5 neutral samples of actual feedback text, and combines them with the aggregate stats into a single prompt asking the model to respond with **only** a JSON object matching an exact schema (`response_format={"type": "json_object"}`) is also passed to the Groq API to enforce this).

Failure handling

The call is wrapped in a `try/except`. If the API call fails, or the response can't be parsed, the function falls back to `DEFAULT_INSIGHT` with a summary explaining the AI insight is "temporarily unavailable - showing live statistics only." The function is documented to **never raise** — a page render should never break just because the LLM had a bad moment.

Caching — `get_cached_insight()`

This is the piece that keeps the app fast. An `AIInsight` row is looked up by `(user_id, scope)`. If one exists and its stored `feedback_count` still matches the current count (and no manual refresh was requested), the cached fields are returned directly — **no LLM call, and no full feedback-table fetch**, since the `fetch_feedbacks` callback is only invoked on an actual cache miss. Only when the count has changed (new feedback was added or removed) does it regenerate, save the new result, and update the count. Users can also disable AI insights entirely from Settings, in which case a static "disabled" message is stored instead of calling the LLM.

The practical effect: viewing the dashboard, reports, or PDF export is cheap on every normal page load, and the relatively slow/costly LLM call only happens the first time new data appears, or when a user explicitly clicks "Regenerate" (the `/refresh_ai` route).

9. Authentication & Security

Registration

- Passwords must be 8+ characters and include an uppercase letter, lowercase letter, digit, and special character (`is_strong_password()`).
- Names are validated to be letters/spaces only, and reject obvious keyboard-mash input: no letter may repeat more than 3 times, and no word may repeat within the name (`is_valid_name()`).
- Phone numbers, if provided, must be exactly 10 digits (`is_valid_phone()`).
- Email is optional but, if given, must look like an email and be unique — it is the lookup key for password reset, so silently allowing a duplicate would create two accounts contending for the same reset link.
- Passwords are hashed with `pbkdf2:sha256` via Werkzeug, never stored in plaintext.
- Every self-registered account starts with `role="user"`. There is no admin code on the public form — promotion only happens from the Admin panel by an existing admin (`/admin/set_role`), closing off what used to be a hardcoded "SECRET123" self-promotion field.

Login & brute-force protection

A simple in-memory dict, `login_attempts`, counts failed attempts per client IP. After 5 failures, further attempts from that IP are blocked with a flash message until... (the counter resets to 0 only on a successful login for that IP). This is process-local, in-memory state — adequate for a single-process deployment, not a distributed rate limiter.

Password reset

A single active, database-backed token per user (issuing a new one invalidates any previous one, unlike an in-memory dict, so it survives a server restart or multiple worker processes). The token is a URL-safe random string (`secrets.token_urlsafe(32)`) with a 60-minute expiry. The "forgot password" endpoint always returns the identical flash message regardless of whether the email matched an account, mail is configured, or sending succeeded — otherwise the endpoint would let an attacker enumerate which emails have registered accounts.

Access control

Every feedback-owning route checks `_owns_or_admin()`: a non-admin can only view/edit/ delete their own rows; an admin can act on anything. Admin-only routes (`/admin`, `/admin/set_role`, `/delete_feedback`) explicitly check `current_user.role == 'admin'` and `abort(403)` otherwise. The system also refuses to demote the last remaining admin account, so the app can never end up with zero admins.

CSRF & other protections

Flask-WTF's `CSRFProtect` is applied to the whole app; the small number of pure-JSON fetch endpoints are individually exempted. File uploads (CSV import, avatar) are constrained: avatars are limited to PNG/JPG/JPEG/WEBP extensions and saved under a `secure_filename()`-sanitized name; CSV import is capped at 5,000 rows per file and the whole request body is capped at 8MB (`MAX_CONTENT_LENGTH`).

10. Feature Walkthrough by Route

Dashboard — `GET/POST /dashboard`

On GET, resolves the current "scope" (a user's own data, or — for admins — all data via `?view=all`), fetches/regenerates the cached AI insight, and renders the page. On POST it handles two different form submissions on the same route: a CSV file upload (routed to `_import_csv_rows()`) or a single manual feedback entry, both of which end in `classify_and_log()` and a database commit.

Live sentiment preview — `POST /live_sentiment`

Called on every keystroke from the dashboard's textarea via `fetch()`. CSRF-exempt because it is a lightweight, read-only preview with no persisted side effect. Returns sentiment, confidence, and emotion for whatever text has been typed so far.

Feedback submission API — `POST /api/submit_feedback`

A JSON equivalent of the manual dashboard form, for programmatic/API-style submission. Same validation and classification path.

Dashboard data API — `GET /api/dashboard_data`

Powers the dashboard's 30-second auto-refresh: totals, sentiment split, top emotion, a bounded sentiment trend window (last `TREND_WINDOW` = 200 points, one value per feedback row, 0/0.5/1 for Negative/Neutral/Positive), the cached self-consistency evaluation, and the 10 most recent feedback rows. Uses SQL `GROUP BY` aggregation rather than pulling every row into Python.

Analytics — `GET /analytics` and `GET /api/analytics_data`

`compute_analytics()` is shared by both the page render and the polling API: sentiment counts, emotion counts, per-service counts, a bounded trend window, and an overall CSAT-style score.

Customers — `GET /customers`

A searchable, filterable, paginated table of a user's own feedback (search by name/service substring, filter by exact sentiment or service, 10 rows per page).

Reports & export — `/reports`, `/download_report`, `/export_csv`

`build_report_context()` is the single source of truth for report data, shared by both the on-screen Reports page and the downloadable PDF, so the two views can never drift apart. The PDF path additionally renders a matplotlib donut chart to an in-memory PNG (never touching disk, so concurrent requests can't clobber each other's chart image) and feeds the rendered HTML through `xhtml2pdf`. `/export_csv` streams a user's raw feedback rows as a CSV attachment using Python's built-in `csv` module.

Admin panel — `/admin`, `/delete_feedback`, `/admin/set_role`

Lists all users and a paginated feedback table. Site-wide totals are computed with direct `Feedback.query.count()/filter_by(...).count()` calls rather than measuring the current page of paginated results, which would silently undercount everything past page 1.

Settings — `/settings`, `/upload_avatar`, `/delete_account`, `/clear_data`

Username/email/password changes (guarded by re-entering the current password), the `ai_enabled`/`email_notify` toggles, avatar upload (old avatar file is deleted from disk after a successful replace), full data wipe, and full account deletion (feedback rows cascade via the FK relationship, then the user row itself is deleted and the session logged out).

11. Frontend Layer

`templates/base.html` defines the shared shell: navigation, sidebar, and the dark/light theme toggle; every other template extends it with `{% extends "base.html" %}` and fills in a content block. `_macros.html` holds small reusable snippets, such as the KPI card used repeatedly across the dashboard and analytics pages.

Styling is done with **Tailwind CSS** utility classes directly in the templates, rather than large hand-written stylesheet files — layout, color, spacing and responsive behavior are expressed inline on each element.

JavaScript files (`static/js/`)

File	Purpose
<code>custom-select.js</code>	A styled dropdown/select component used for filters (sentiment, service) without relying on the browser's native <code><select></code> styling.
<code>kpi-count.js</code>	Animates the KPI numbers on the dashboard counting up to their target value, and drives the periodic re-fetch from <code>/api/dashboard_data</code> .
<code>theme.js</code>	Persists and applies the light/dark theme choice.

Chart.js, **lucide** (icons), and **three.js** (background visual effects) are self-hosted under `static/vendor/` rather than loaded from a CDN, and are given a 30-day immutable cache header since their content is pinned by the committed file version.

12. PDF Report Pipeline

The downloadable report follows this sequence in `download_report()`:

1. Resolve scope and fetch feedback rows (same `_dashboard_scope()` helper used elsewhere).
2. Fetch/regenerate the cached AI insight via `get_cached_insight()`.
3. Build the shared report context with `build_report_context()` (stats + insight + top negative/positive examples).
4. Render a sentiment donut chart to a base64 PNG in memory with `matplotlib` (`render_sentiment_chart_base64()`).
5. Render the `report_pdf.html` Jinja2 template with all of the above into a plain HTML string.
6. Feed that HTML into `xhtml2pdf.pisa.CreatePDF()`, which converts it into a PDF byte stream held entirely in memory (a `BytesIO` buffer).
7. Stream the buffer back to the browser as a file download.

Because everything happens in memory (no temp files on disk), concurrent requests from different users can't interfere with each other's report generation.

13. Running & Configuring the Project

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Configure environment (copy .env.example to .env and fill in values)
cp .env.example .env

# 3. Run the app - pending migrations are applied automatically on startup
python app.py

# App is now available at:
http://127.0.0.1:5000
```

Required / optional environment variables

Variable	Required	Purpose
GROQ_API_KEY	Yes	The app raises on startup without it — used by the AI insight engine.
SECRET_KEY	No	Flask session signing key; a random one is generated if unset (sessions won't survive a restart in that case).
DATABASE_URL	No	Defaults to a local SQLite file; set for PostgreSQL in production.
MAIL_SERVER / MAIL_PORT / MAIL_USE_TLS / MAIL_USERNAME / MAIL_PASSWORD / MAIL_DEFAULT_SENDER	No	SMTP settings for password-reset email; without credentials, reset requests are logged but not sent.

Changing the database schema

```
flask db migrate -m "describe the change"
flask db upgrade
```

The first command inspects the difference between `models.py` and the current database and writes a new file under `migrations/versions/`; the second applies it. In this project that second step also happens automatically the next time `python app.py` starts.

14. API Quick Reference

Method & Path	Auth	Purpose
POST /live_sentiment	Login required	Real-time sentiment/emotion preview while typing
POST /api/submit_feedback	Login required	Create a feedback row via JSON
GET /api/dashboard_data	Login required	Dashboard KPIs, trend, evaluation metrics, recent rows

GET /api/analytics_data	Login required	Sentiment/emotion/service breakdowns
GET /download_report	Login required	Generates and downloads the PDF report
GET /export_csv	Login required	Downloads the user's own feedback as CSV
GET /refresh_ai	Login required	Forces regeneration of the cached AI insight
POST /admin/set_role/<user_id>	Admin only	Promote/demote a user

15. Glossary of Terms

Term	Meaning in this project
ORM	Object-Relational Mapper — lets Python classes (<code>models.py</code>) represent database tables, so code reads/writes rows as objects instead of raw SQL.
Migration	A versioned, scripted change to the database schema, managed by Alembic/Flask-Migrate.
CSRF	Cross-Site Request Forgery — an attack where a malicious site tricks a logged-in user's browser into submitting a request; prevented here with per-form tokens via Flask-WTF.
Polarity	TextBlob's -1..+1 sentiment score for a piece of text; the raw signal behind the Positive/Negative/Neutral label.
RBAC	Role-Based Access Control — here, the two-value <code>role</code> field ("user"/"admin") that gates access to admin-only routes.
LLM	Large Language Model — the Groq-hosted model used to turn feedback data into a narrative business insight.
Self-consistency check	Re-running the current classifier over already-stored rows and measuring agreement with what was saved — not the same as accuracy against human-labeled ground truth.
Cache invalidation (here)	An AI insight or evaluation result is considered stale, and regenerated, only when the underlying feedback row count has changed.

16. Suggested Learning Path

A practical order for reading the codebase for the first time:

1. **`models.py`** first — it's short, and every route makes more sense once you know the shape of `User`, `Feedback`, `AuditLog`, and `AIInsight`.
2. **`app.py` top section** (through the login manager setup) — app configuration, extensions, and error handling.
3. **`get_sentiment / detect_emotion / classify_and_log`** — the classification core that almost everything else calls into.
4. **`generate_ai_insights / get_cached_insight`** — the LLM integration and its caching strategy.
5. **The route functions themselves**, in the order they appear in the file (register → login → password reset → dashboard → analytics → customers → reports → admin → settings).
6. **`templates/base.html`** and one leaf template (e.g. `dashboard.html`) side by side with the route that renders it, to see how server-side data becomes the on-screen UI.
7. **`static/js/kpi-count.js`** alongside `/api/dashboard_data` to see the full round trip of the dashboard's live auto-refresh.

A few good exercises to build confidence with the codebase:

- Add a new emotion keyword rule to `detect_emotion()` and confirm it shows up in `/live_sentiment`.

- Add a new field to Feedback in `models.py`, then run `flask db migrate / flask db upgrade` to see a migration get generated and applied.
- Trace a single dashboard page load from the incoming request in `dashboard()` all the way to the JSON returned by `/api/dashboard_data` and the JS that renders it.
- Temporarily lower `LOW_CONFIDENCE_THRESHOLD` and watch the audit log fill up faster, to understand what "confidence" actually measures here.